

Energy — metamodel pipeline

This notebook builds the metamodel for the energy complex — WTI crude, heating oil, RBOB gasoline and Henry Hub natural gas. Energy is the awkward, and most rewarding, corner of the universe. The instruments do not share a single signal character: crude behaves unlike the refined products, and natural gas is thin enough that it must be modelled in a pool rather than alone. Despite that heterogeneity, energy is the engine of the pooled strategy's out-of-sample return. The sections trace how the signal, the labelling and the per-instrument model choices combine into that contribution.

Universe. `cl1s`, `ho1s`, `rb1s`, `ng1s` (4 contracts).

Reading guide. The notebook runs top to bottom against committed artifacts; no model is retrained. Each stage loads its result files and embeds the figures already produced by the pipeline. Section order mirrors the marking scheme in `reports/harry/00-context.md`: data, signal, labelling, features, model selection, cluster importance, evaluation.

Source of truth. Where a number appears it is loaded from a file in this cell's output, not transcribed from prose; the few places where the committed data and the written reports disagree are flagged inline.

```
In [1]: CLASS = 'energy'
INSTS = ['cl1s', 'ho1s', 'rb1s', 'ng1s']
NAMES = {'cl1s': 'WTI crude (CL)', 'ho1s': 'Heating oil (HO)', 'rb1s':
'RB0B gasoline (RB)', 'ng1s': 'Henry Hub natural gas (NG)'}

from pathlib import Path
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from IPython.display import Image, display, Markdown

def _repo_root() -> Path:
    here = Path.cwd().resolve()
    for q in [here, *here.parents]:
        if (q / "pyproject.toml").exists():
            return q
    return here

REPO = _repo_root()
OUT = REPO / "src/stml/new_work/outputs"
RESULTS = REPO / "results"
REPORTS = REPO / "reports"
DATA = REPO / "data"
```

```

def show(path, width=1000):
    p = Path(path)
    if p.exists():
        display(Image(str(p), width=width))
    else:
        print(f"[missing figure] {p}")

def load(path) -> pd.DataFrame:
    p = Path(path)
    if p.exists():
        return pd.read_csv(p)
    print(f"[missing data] {p}")
    return pd.DataFrame()

def table(df, n=None, caption=None):
    if df is None or len(df) == 0:
        print("[empty table]")
        return
    d = df.head(n) if n else df
    sty = d.style.format(precision=4).hide(axis="index")
    if caption:
        sty = sty.set_caption(caption)
    display(sty)

```

1. Data and cleaning

The price history runs from 1990, while the primary signal is released only over 2020-01-03 to 2022-06-30; the sealed test period sits beyond the embargo. Loading goes through `stm1.io`, and the missing-data policy — keep zero-volume weekday settles, drop weekend and out-of-bounds rows — is documented in `reports/missing-data-report.md` and implemented in `stm1.na_checks`. The coverage table below is computed directly from `data/ohlcvc_data.csv` for this class's contracts.

```

In [2]: ohlcv = load(DATA / "ohlcvc_data.csv")
sub = ohlcv[ohlcv["instrument"].isin(INSTS)]
cov = (sub.groupby("instrument")
        .agg(first_date=("date", "min"), last_date=("date", "max"),
        n_rows=("date", "size"))
        .reindex(INSTS).reset_index())
table(cov, caption=f"{CLASS.title()} – OHLCV coverage
(data/ohlcvc_data.csv)")

```

Energy — OHLCV coverage (data/ohlcv_data.csv)

instrument	first_date	last_date	n_rows
cl1s	1990-01-02	2022-06-30	8171
ho1s	1990-01-02	2022-06-30	8169
rb1s	1990-01-02	2022-06-30	8170
ng1s	1990-04-04	2022-06-30	8104

2. Signal characterisation

Energy breaks the tidy contrarian story seen elsewhere. Crude is the outlier: its trailing-return correlation is close to zero and its hit rate is the highest in the universe, so it is better treated as a near-momentum special case than lumped with the refined products. Heating oil and gasoline reverse more conventionally, while natural gas has so few clean bets that its statistics are directional at best. This is why the modelling stage pools the thin names rather than fitting each in isolation.

```
In [3]: sd = load(RESULTS / "harry/signal_direction.csv")
if not sd.empty:
    want = ["instrument", "n_bets", "corr_fwd_1", "corr_fwd_5",
"corr_trail_1", "corr_contemp_0"]
    cols = [c for c in want if c in sd.columns]
    rows = sd[sd["instrument"].isin(INSTS)]
    [cols].set_index("instrument").reindex(INSTS).reset_index()
    table(rows, caption=f"{CLASS.title()} – signal direction
(results/harry/signal_direction.csv)")
```

Energy — signal direction (results/harry/signal_direction.csv)

instrument	n_bets	corr_fwd_1	corr_fwd_5	corr_trail_1	corr_contemp_0
cl1s	422	0.1207	0.0859	0.0029	0.0005
ho1s	63	0.0357	-0.0422	-0.1143	-0.1548
rb1s	628	0.0654	-0.0269	-0.1025	-0.0695
ng1s	124	0.0933	0.0580	-0.0400	-0.0017

3. Labelling

Each signal is turned into a binary meta-label by the triple-barrier method of López de Prado (2018, ch. 3): an upper (profit-taking) and lower (stop-loss) barrier are placed in units of a causal volatility estimate, a vertical barrier caps the holding period, and the label records whether taking the bet would have paid. Overlapping events are down-weighted by average uniqueness (López de Prado, 2018, ch. 4).

The submitted metamodel is trained and scored on the **team-provided labels** in `data/meta/triple_barrier_labels.csv`. The barrier geometry is tuned per instrument and held constant within each: the stop-loss sits at one quarter of the volatility estimate for every contract (`sl = 0.25`), while the profit-taking multiple and the vertical-barrier horizon are set per instrument — some contracts take a tight one-day barrier, others a wider profit target held for two to four weeks. The exact geometry is read from the data and shown per instrument below rather than quoted, because the values differ across the universe.

Provenance is not assumed: every locked out-of-sample prediction in `metamodel_predictions.csv` matches these labels exactly — 1373 of 1373 rows agree on the binary label after joining on instrument and date — so this file, not any other, is the label set the model was built on.

Provenance note. An earlier exploration documented in `reports/harry/02-labels.md` and persisted to `results/harry/events.csv` used a uniform ten-day barrier; it produces the same number of events (one per signal date) but different labels, and it was superseded by the per-instrument team labels used here. The report is retained as a record of that exploration.

```
In [4]: lab = load(DATA / "meta/triple_barrier_labels.csv")
lab = lab[lab["instrument"].isin(INSTS)]
if not lab.empty:
    # Per-instrument barrier geometry + event statistics, straight from the
    # team labels.
    g = (lab.groupby("instrument")
         .agg(pt=("pt", "first"), sl=("sl", "first"), h=("h", "first"),
              n_events=("label", "size"), label1_share=("label",
                                                         "mean"),
              mean_sigma=("sigma", "mean"))
         .reindex(INSTS).reset_index())
    table(g, caption=f"{CLASS.title()} - per-instrument barriers and labels
(data/meta/triple_barrier_labels.csv)")
    if "partition" in lab.columns:
        part = (lab.groupby(["instrument", "partition"]).size()
                 .unstack(fill_value=0).reindex(INSTS).reset_index())
        table(part, caption="Events per train / validation / test
partition")
    print(f"Total labelled events for {CLASS}: {len(lab):,} | label==1
share: {lab['label'].mean():.3f}")
```

Energy — per-instrument barriers and labels

(data/meta/triple_barrier_labels.csv)

instrument	pt	sl	h	n_events	label1_share	mean_sigma
cl1s	0.2500	0.2500	1	421	0.5986	0.0321
ho1s	0.2500	0.2500	1	63	0.6667	0.0260
rb1s	2.5000	0.2500	15	613	0.2626	0.0294
ng1s	0.2500	0.2500	1	124	0.5242	0.0527

Events per train / validation / test
partition

instrument	test	train	val
cl1s	88	229	104
ho1s	2	34	27
rb1s	110	377	126
ng1s	56	28	40

Total labelled events for energy: 1,221 | label==1 share: 0.426

4. Features

Predictors are grouped into causal families, all computed at the signal date and predictive of the next-day label, so none peeks into the future. The micro families (`reports/harry/03-features.md`) cover the signal's own trajectory, conditional risk, information-theoretic dependence, fixed microstructure proxies, cross-asset structure, an optional wavelet decomposition and a concept-drift discriminator. A macro block of thirty-one rolling, standardised series (`reports/harry/03-6-macro-features.md`) adds volatility, rates, credit, dollar, commodity-fundamental and growth context. Two hidden-Markov regime blocks — per-instrument volatility (`hmm_vol`) and a global risk-on/risk-off state (`hmm_macro`) — are fit only on pre-sample data and then frozen, so they are train-fitted rather than leaking. Features are tagged `E` (purely causal) or `TF` (train-fitted on pre-sample) accordingly.

```
In [5]: # The feature matrix is built by the pipeline; here we confirm the frozen
HMM
# regime features are present for this class's instruments.
for fn in ["features_hmm_vol.csv", "features_hmm_macro.csv"]:
    f = load(REPO / "src/stml/new_work" / fn)
    if not f.empty and "instrument" in f.columns:
        n = f[f["instrument"].isin(INSTS)].shape[0]
        print(f"{fn}: {n:,} rows for {CLASS} instruments | columns:
{list(f.columns)}")
    elif not f.empty:
        print(f"{fn}: {f.shape[0]:,} rows | columns: {list(f.columns)}")
```

```
features_hmm_vol.csv: 2,512 rows for energy instruments | columns: ['date',  
'instrument', 'hmm_vol_p0_calm', 'hmm_vol_p2_turbulent',  
'hmm_vol_next_turbulent', 'hmm_vol_entropy']  
features_hmm_macro.csv: 2,580 rows for energy instruments | columns:  
['date', 'instrument', 'hmm_macro_p0', 'hmm_macro_next_riskoff',  
'hmm_macro_entropy']
```

5. Model selection

Each instrument runs a horse race across penalised logistic regression, a random forest, gradient boosting and a small multi-task network, scored by combinatorial purged cross-validation (six groups, two held out, fifteen paths) with per-instrument purging and embargo. Feature variants (full, pruned, reduced) are compared, and the simplest variant within one cross-path standard error of the best is locked **before** the out-of-sample window is touched. The cross-validation runs on the training partition only; the sealed test (after 2021-10-20) is scored once, with the locked choice fixed.

The per-instrument choices follow the signal's heterogeneity. Crude takes a standalone penalised logistic; the refined products and gas borrow strength from pooled fits (the crude-heating-oil pool and the all-energy pool) because their own event counts are too small to support an independent model. Gasoline's locked variant is deliberately minimal — a check on whether its apparent edge is anything more than the raw signal plus a credit spread. The locked picks and their out-of-sample AUCs below should be read with the pooling in mind: a pooled score is a statement about the group, not the single name.

```
In [6]: mc = OUT / "model_comparison" / CLASS  
sel = load(OUT / "model_comparison/selection_table.csv")  
locked = load(mc / "locked_picks.csv")  
cpcv = load(mc / "cpcv_results.csv")  
oos = load(mc / "oos_results.csv")  
  
if not sel.empty:  
    # NB: selection_table's own 'n_events' is a derived training-sample  
    # count, not the  
    # triple-barrier event count reported in section 3, so it is omitted  
    # here to avoid a  
    # misleading clash.  
    keep = [c for c in ["instrument", "best_model", "best_auc", "lower_ci",  
"signal"]  
            if c in sel.columns]  
    sc = sel[sel["instrument"].isin(INSTS)]  
    [keep].set_index("instrument").reindex(INSTS).reset_index()  
    table(sc, caption=f"{CLASS.title()} – champion model and signal verdict  
(selection_table.csv)")  
  
if not locked.empty:  
    table(locked, caption="Locked feature variant per instrument (1SE rule,  
train-only)")  
  
for inst in INSTS:
```

```

display(Markdown(f"**{NAMES[inst]} - `{inst}` cross-validated
variants**"))
show(mc / f"{inst}_cpcv_chart.png", width=760)

show(mc / "oos_summary_chart.png", width=820)

# Out-of-sample AUC for the locked variants only.
if not oos.empty and not locked.empty:
    lk = dict(zip(locked["inst"], locked["locked_variant"]))
    mask = oos.apply(lambda r: lk.get(r["inst"]) == r["variant"], axis=1)
    keep = [c for c in ["inst", "variant", "auc", "auc_ci_lo", "auc_ci_hi",
"n_test"] if c in oos.columns]
    table(oos[mask][keep], caption="Out-of-sample AUC - locked variants
only")

```

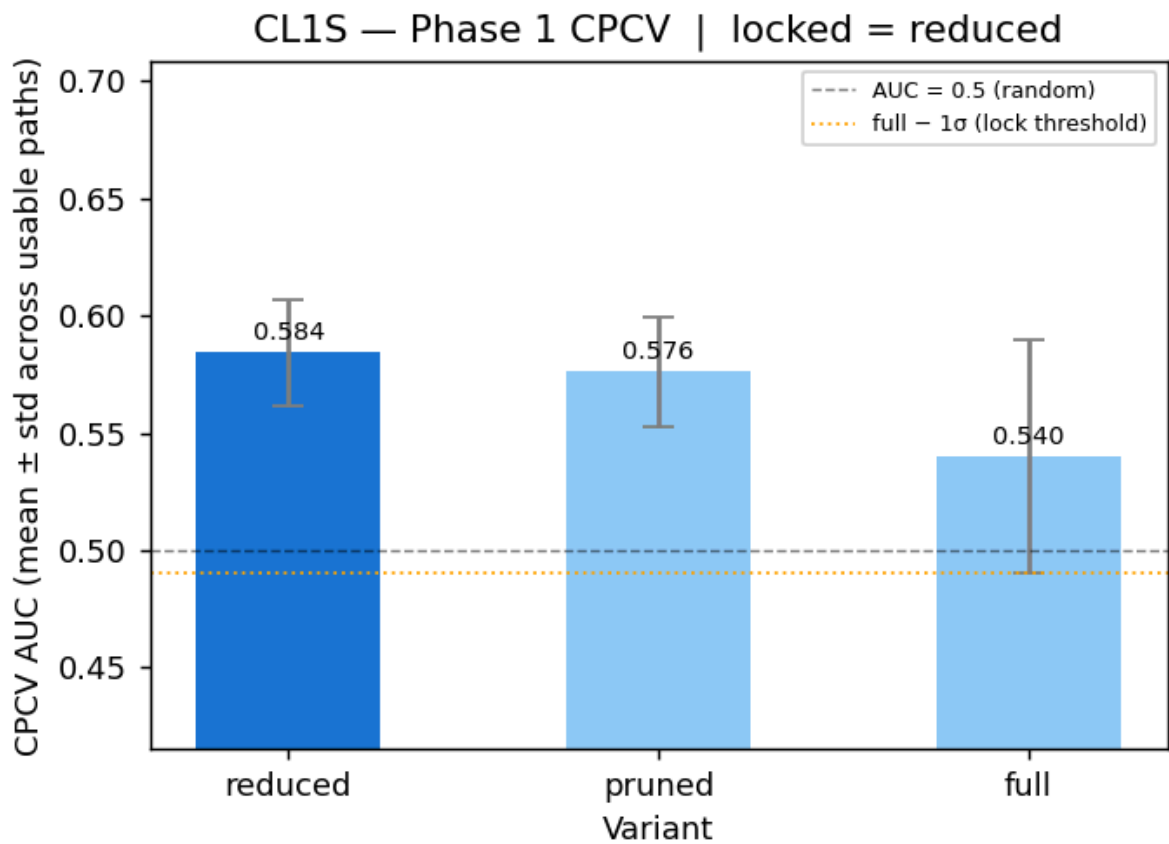
Energy — champion model and signal verdict
(selection_table.csv)

instrument	best_model	best_auc	lower_ci	signal
cl1s	logistic	0.5374	0.4876	False
ho1s	mlp	0.4923	0.3031	False
rb1s	logistic	0.5773	0.5063	True
ng1s	mlp	0.6576	0.5187	True

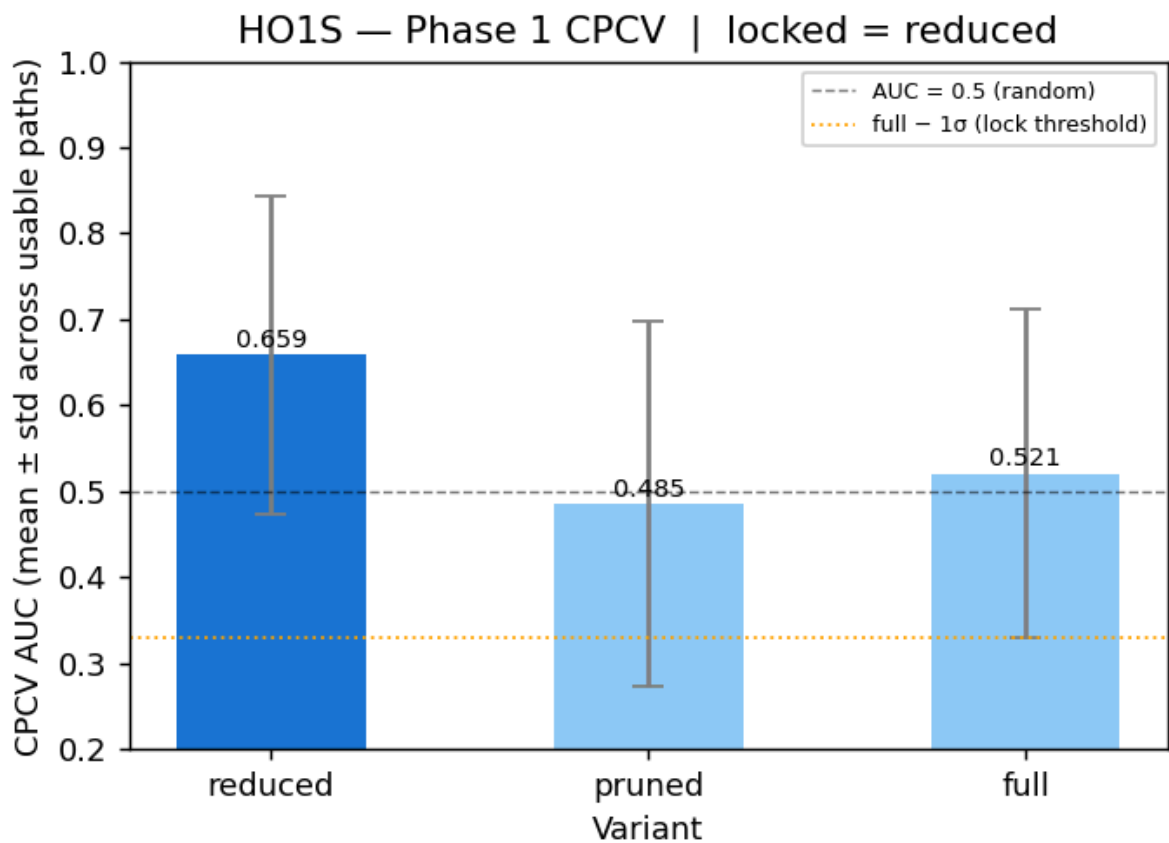
Locked feature variant
per instrument (1SE rule,
train-only)

inst	locked_variant
cl1s	reduced
ho1s	reduced
rb1s	reduced_min
ng1s	reduced

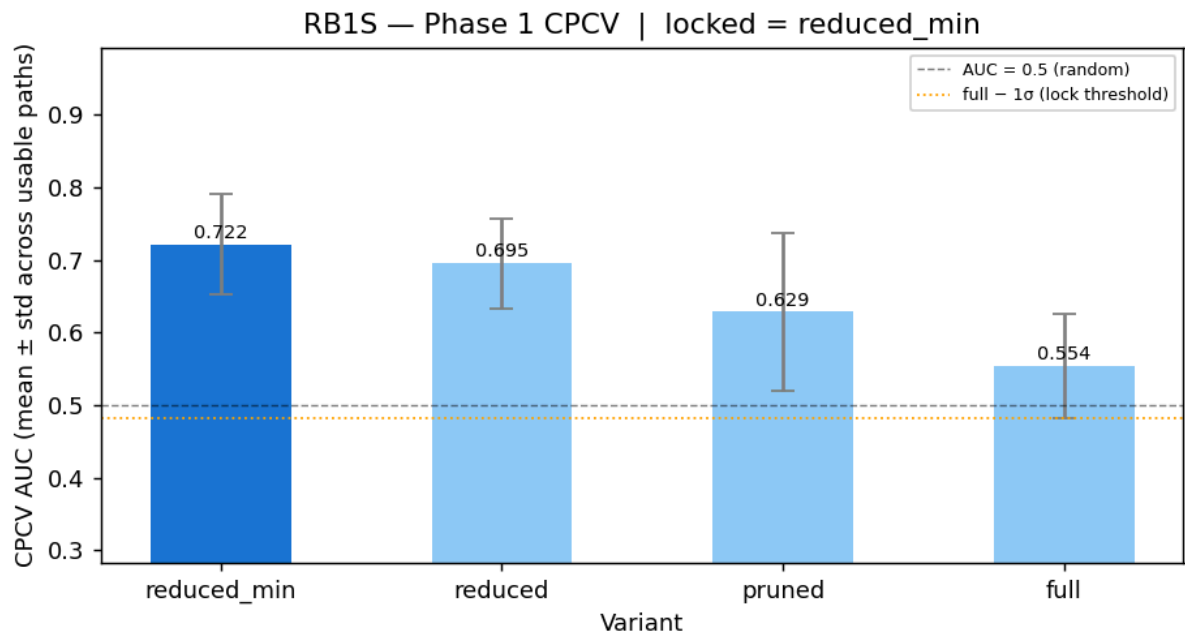
WTI crude (CL) — **cl1s** cross-validated variants



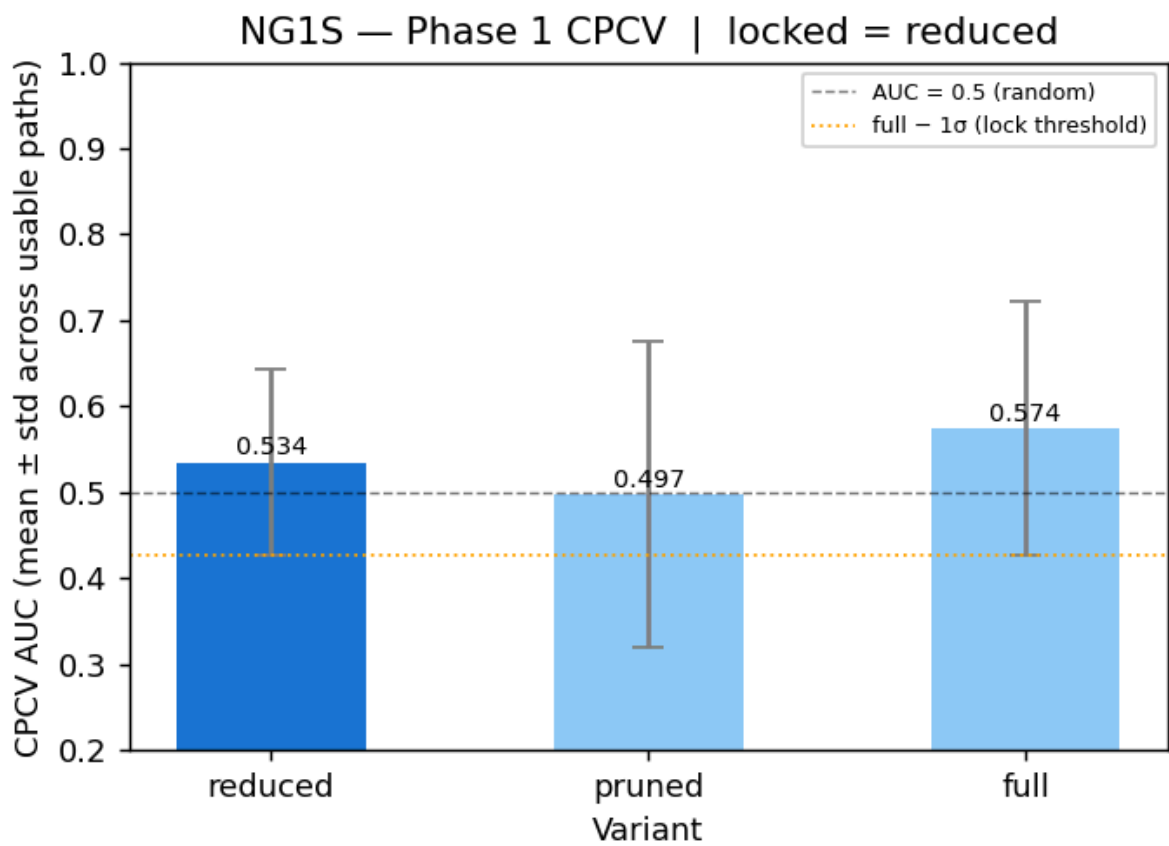
Heating oil (HO) — ho1s cross-validated variants

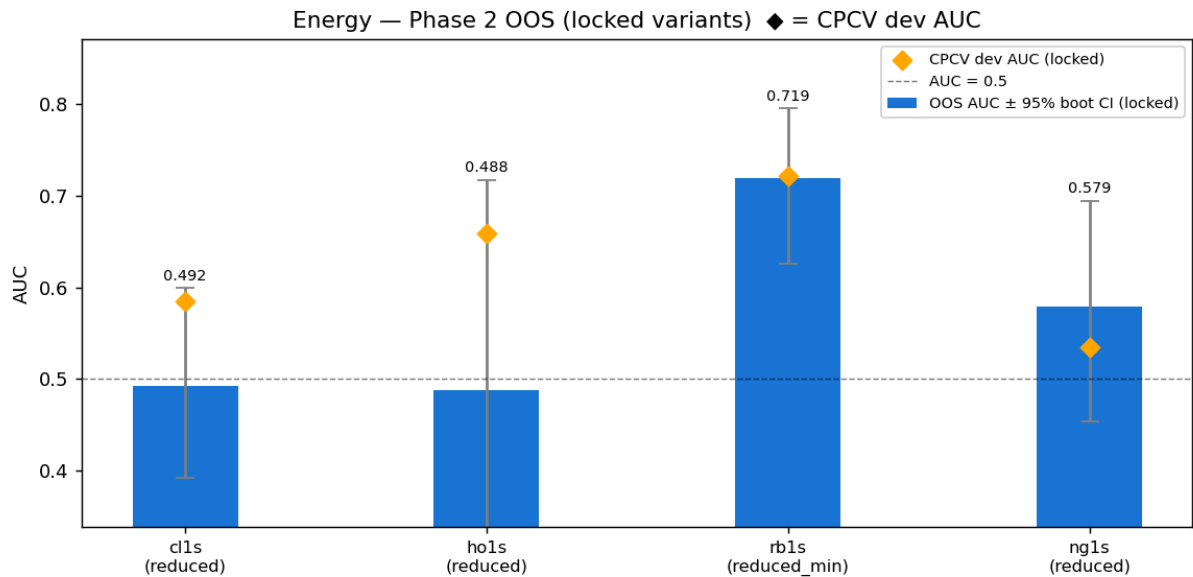


RBOB gasoline (RB) — **rb1s** cross-validated variants



Henry Hub natural gas (NG) — **ng1s** cross-validated variants





Out-of-sample AUC — locked variants only

inst	variant	auc	auc_ci_lo	auc_ci_hi	n_test
cl1s	reduced	0.4924	0.3920	0.5995	127
ho1s	reduced	0.4877	0.2527	0.7172	27
rb1s	reduced_min	0.7191	0.6259	0.7957	159
ng1s	reduced	0.5795	0.4531	0.6944	84

6. Feature importance

Importance is read at the cluster level rather than per raw feature, because the predictors are correlated in blocks and per-feature scores would split credit arbitrarily across substitutes. Features are clustered on a rank-correlation distance; the dendrogram shows the block structure, mean-decrease-accuracy gives each cluster's drop, and a global SHAP or coefficient ranking resolves the within-cluster detail. Logistic champions are read through their coefficients, tree and network champions through SHAP.

Because several energy names are modelled in pools, the importance plots speak to shared structure as much as to any one contract. The commodity-fundamental macro block — inventory surprises and the dollar — earns its place alongside the signal trajectory features, which is the expected fingerprint of a complex driven by physical supply as well as positioning. Crude's standalone importance picture is the most self-contained of the four.

```
In [7]: imp = OUT / "importance"
sel = load(OUT / "model_comparison/selection_table.csv")
model_of = dict(zip(sel.get("instrument", []), sel.get("best_model", [])))
if not sel.empty else {}
```

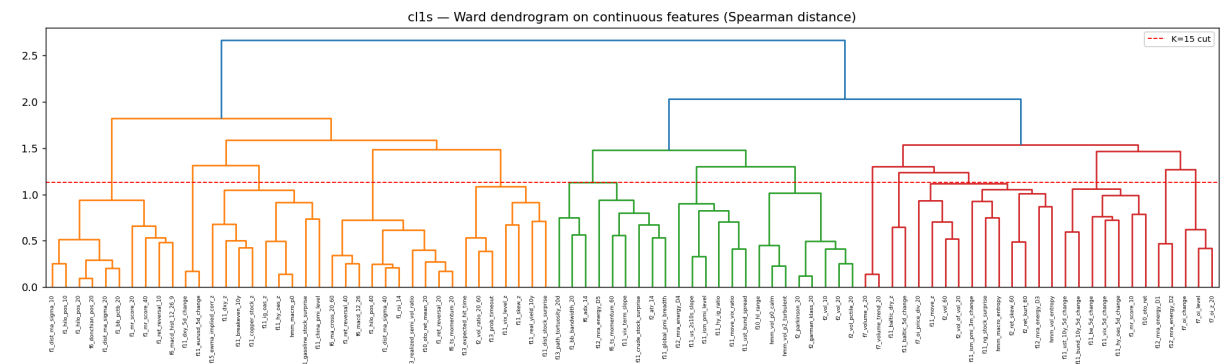
```

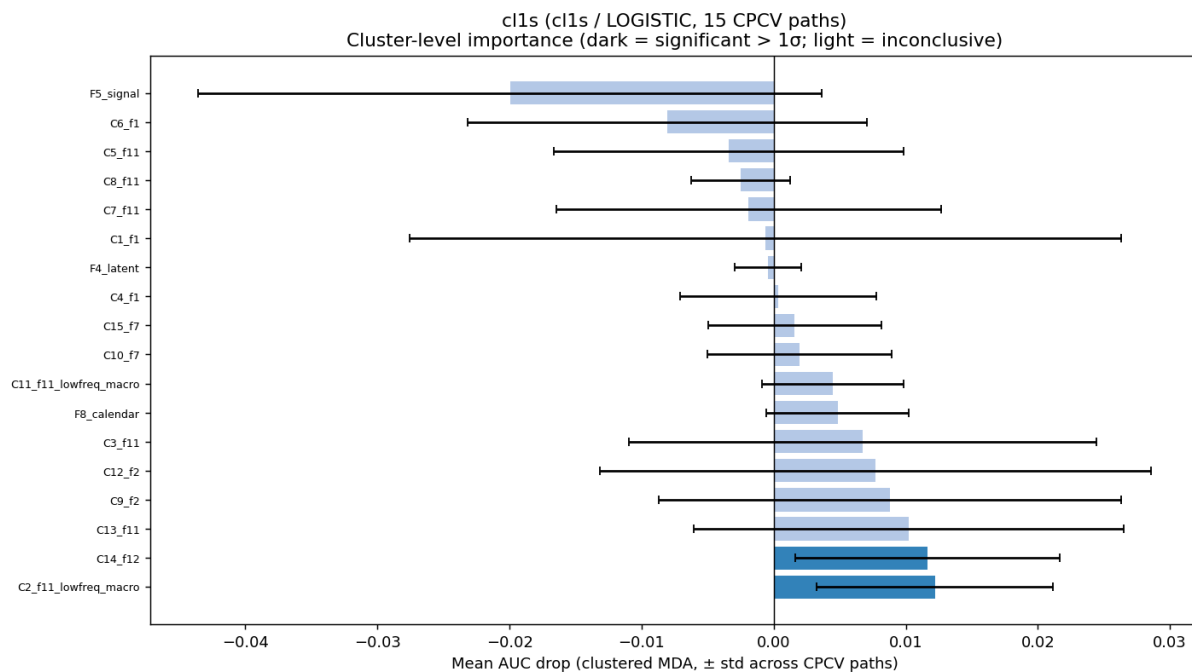
for inst in INSTS:
    d = imp / inst
    display(Markdown(f"### {NAMES[inst]} - `{inst}`"))
    cm = load(d / "champion_meta.csv")
    if not cm.empty:
        table(cm)
    show(d / "dendrogram.png", width=900)
    show(d / "clustered_mda_chart.png", width=820)
    mda = load(d / "clustered_mda_full.csv")
    if not mda.empty:
        table(mda.head(8), caption="Top clusters by mean decrease in accuracy")
    coef_png = d / "global_coef_chart.png"
    if str(model_of.get(inst, "")) == "logistic" and coef_png.exists():
        show(coef_png, width=820)
        table(load(d / "global_coef_summary.csv").head(15), caption="Top logistic coefficients")
    else:
        show(d / "global_shap_chart.png", width=820)
        table(load(d / "global_shap_summary.csv").head(15), caption="Top features by |SHAP|")

```

WTI crude (CL) — cl1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	sign
cl1s	energy	cl1s	logistic	logistic	0.5374	0.0498	0.4876	Fa

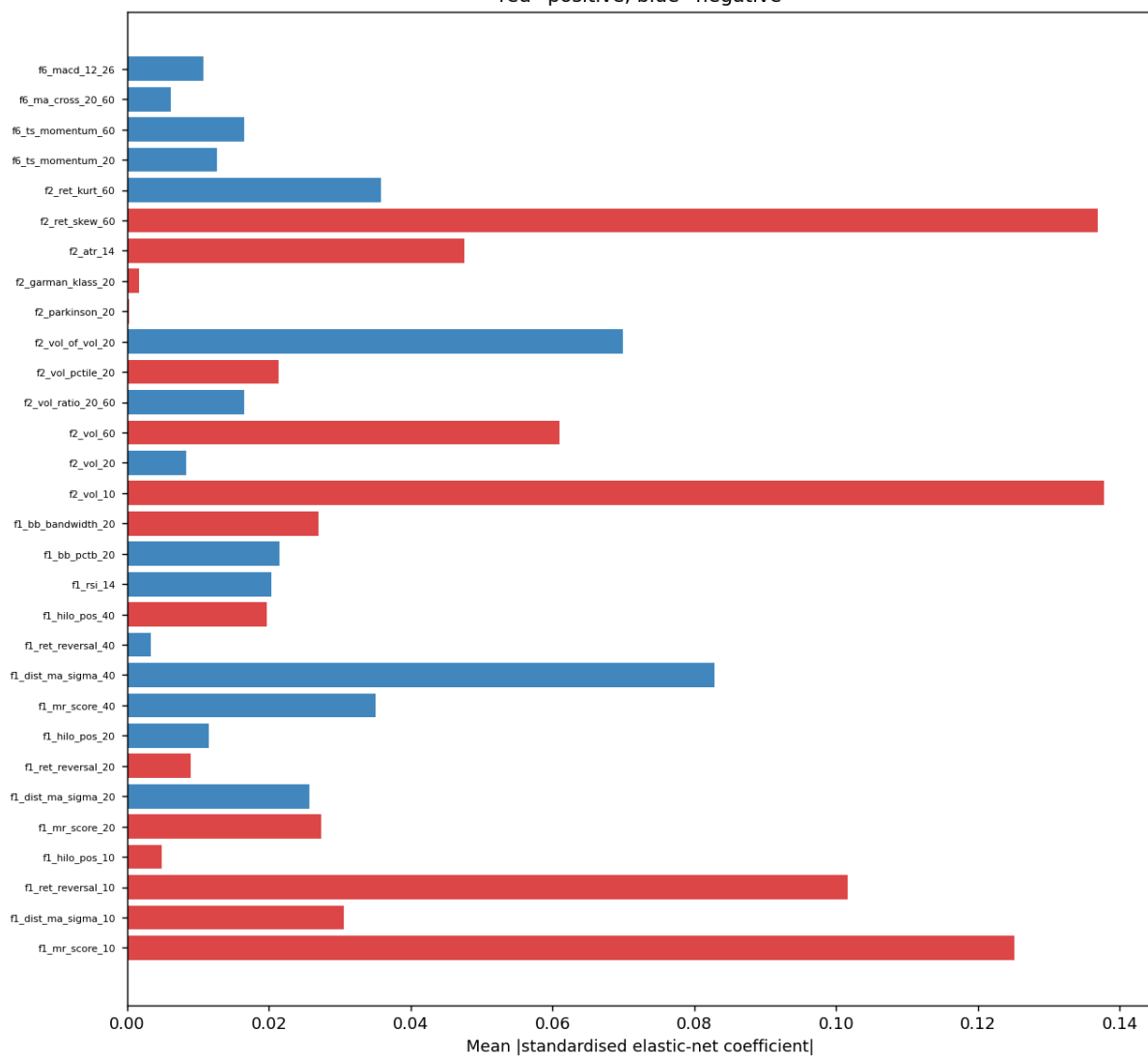




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C2_f11_lowfreq_macro	0.0122	0.0089	True
C14_f12	0.0116	0.0100	True
C13_f11	0.0102	0.0163	False
C9_f2	0.0088	0.0175	False
C12_f2	0.0077	0.0209	False
C3_f11	0.0067	0.0177	False
F8_calendar	0.0049	0.0054	False
C11_f11_lowfreq_macro	0.0045	0.0053	False

cl1s — Global per-feature coefficient (top 30)
red=positive, blue=negative

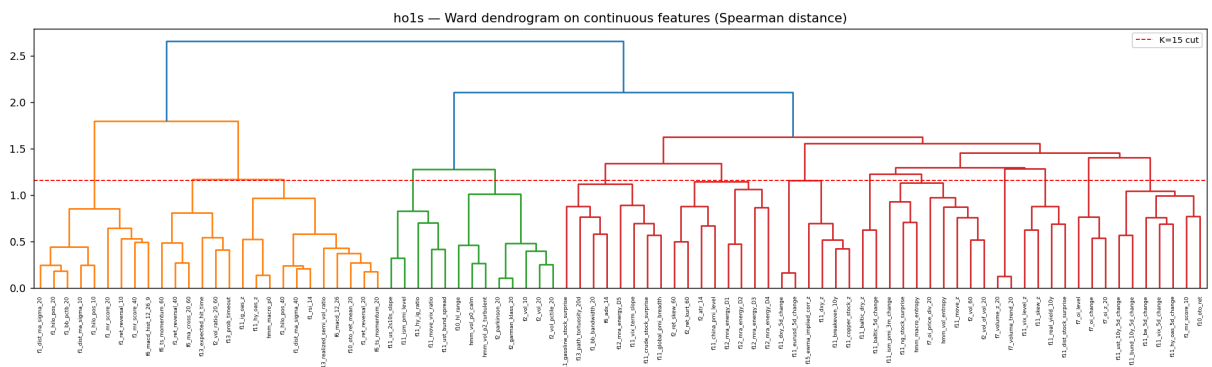


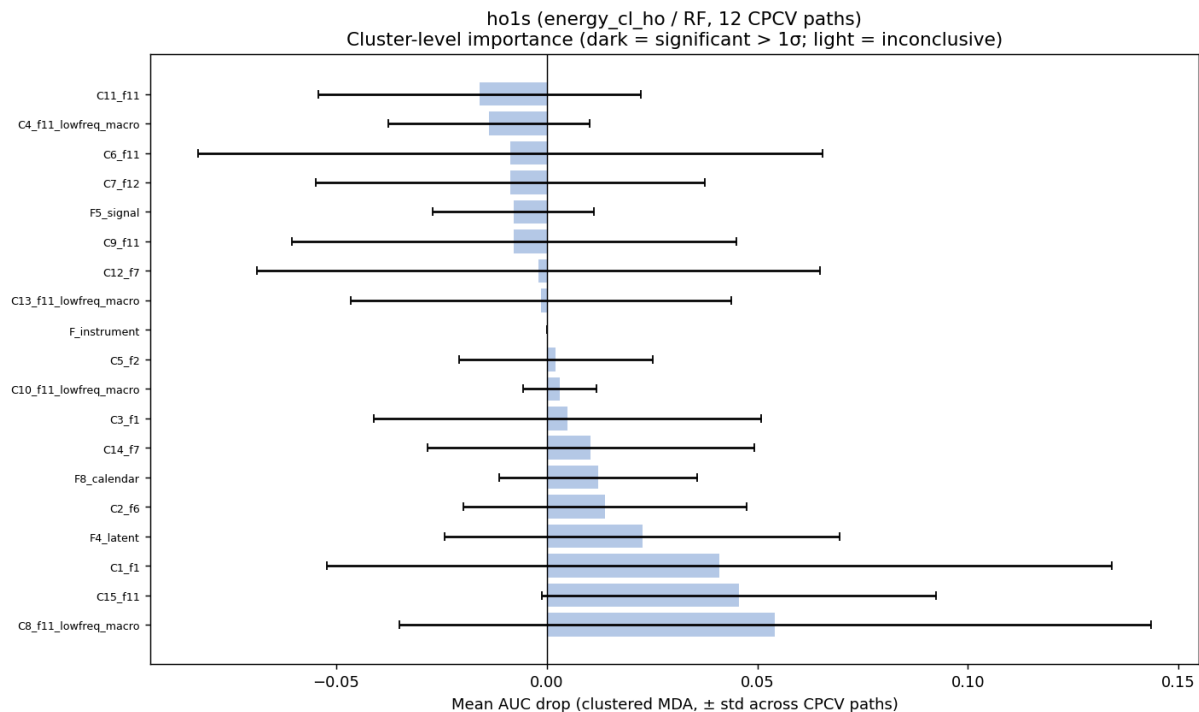
Top logistic coefficients

feature	coef_abs	coef_signed
f11_crude_stock_surprise	0.1433	-0.1433
f2_vol_10	0.1377	0.1377
f2_ret_skew_60	0.1369	0.1369
f11_china_pmi_level	0.1323	-0.1323
f1_mr_score_10	0.1251	0.1251
f8_dow_cos	0.1222	0.1222
f11_dxy_5d_change	0.1182	0.1182
hmm_macro_entropy	0.1109	0.1109
f11_move_z	0.1087	-0.1087
f11_baltic_5d_change	0.1022	0.1022
f1_ret_reversal_10	0.1016	0.1016
f7_oi_change	0.0860	-0.0860
f12_mra_energy_D3	0.0844	-0.0844
f1_dist_ma_sigma_40	0.0828	-0.0828
f12_mra_energy_D1	0.0815	0.0815

Heating oil (HO) — ho1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_c
ho1s	energy	energy_cl_ho	tree	rf	0.4923	0.1892	0.303

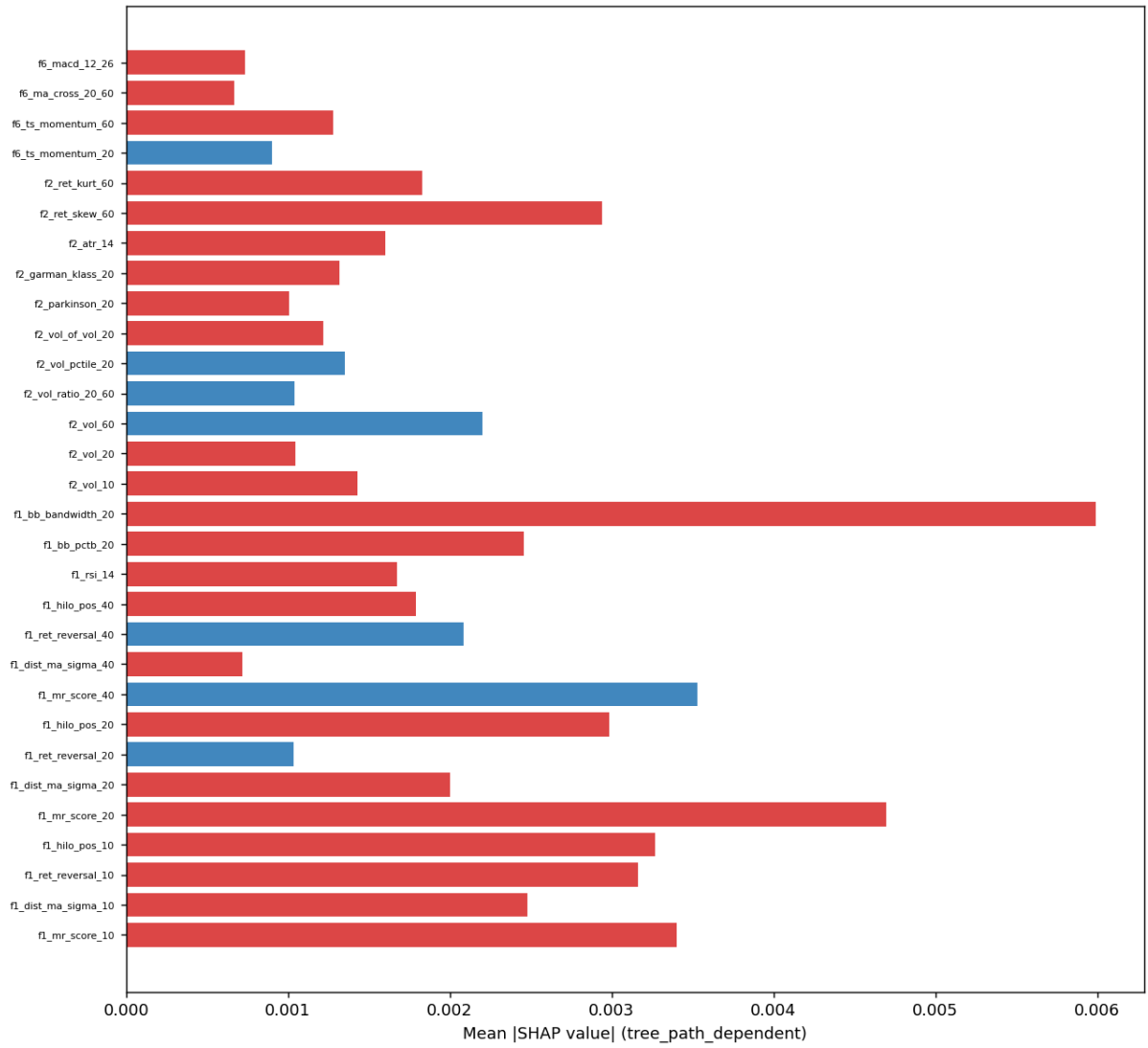




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C8_f11_lowfreq_macro	0.0542	0.0892	False
C15_f11	0.0456	0.0468	False
C1_f1	0.0410	0.0932	False
F4_latent	0.0227	0.0470	False
C2_f6	0.0138	0.0336	False
F8_calendar	0.0122	0.0235	False
C14_f7	0.0104	0.0388	False
C3_f1	0.0049	0.0459	False

ho1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative

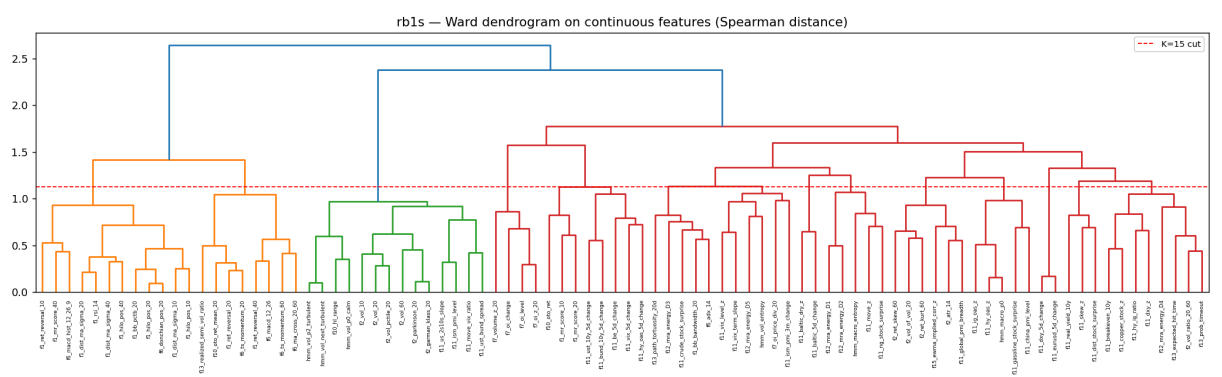


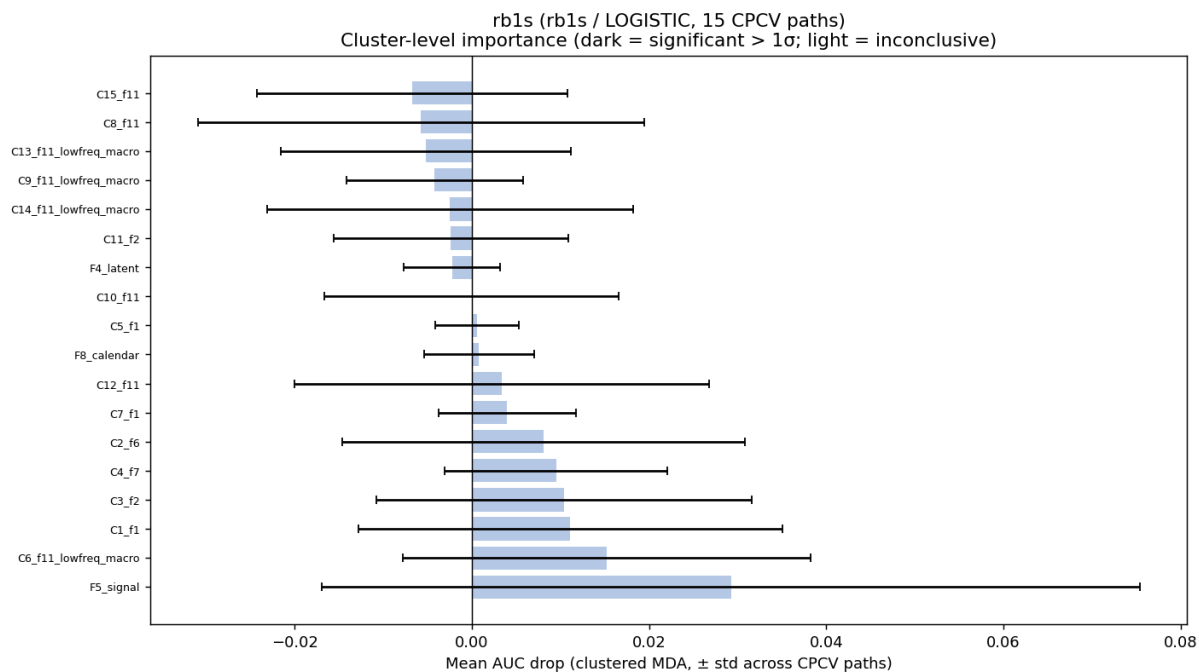
Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f11_vix_term_slope	0.0090	-0.0007	0.0382
f4_pc3	0.0075	-0.0011	0.0255
f1_bb_bandwidth_20	0.0060	0.0010	0.0286
f11_vix_5d_change	0.0057	0.0011	0.0267
f11_real_yield_10y	0.0050	0.0024	0.0164
f11_eurusd_5d_change	0.0049	0.0002	0.0241
f1_mr_score_20	0.0047	0.0020	0.0217
f10_oto_ret	0.0045	0.0025	0.0198
f7_volume_trend_20	0.0038	0.0007	0.0180
f11_skew_z	0.0038	-0.0016	0.0237
f7_volume_z_20	0.0036	0.0008	0.0166
f1_mr_score_40	0.0035	-0.0000	0.0184
f11_dxy_5d_change	0.0035	0.0003	0.0150
f1_mr_score_10	0.0034	0.0014	0.0169
f1_hilo_pos_10	0.0033	0.0024	0.0167

RBOB gasoline (RB) — rb1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	significance
rb1s	energy	rb1s	logistic	logistic	0.5773	0.0710	0.5063	Tr

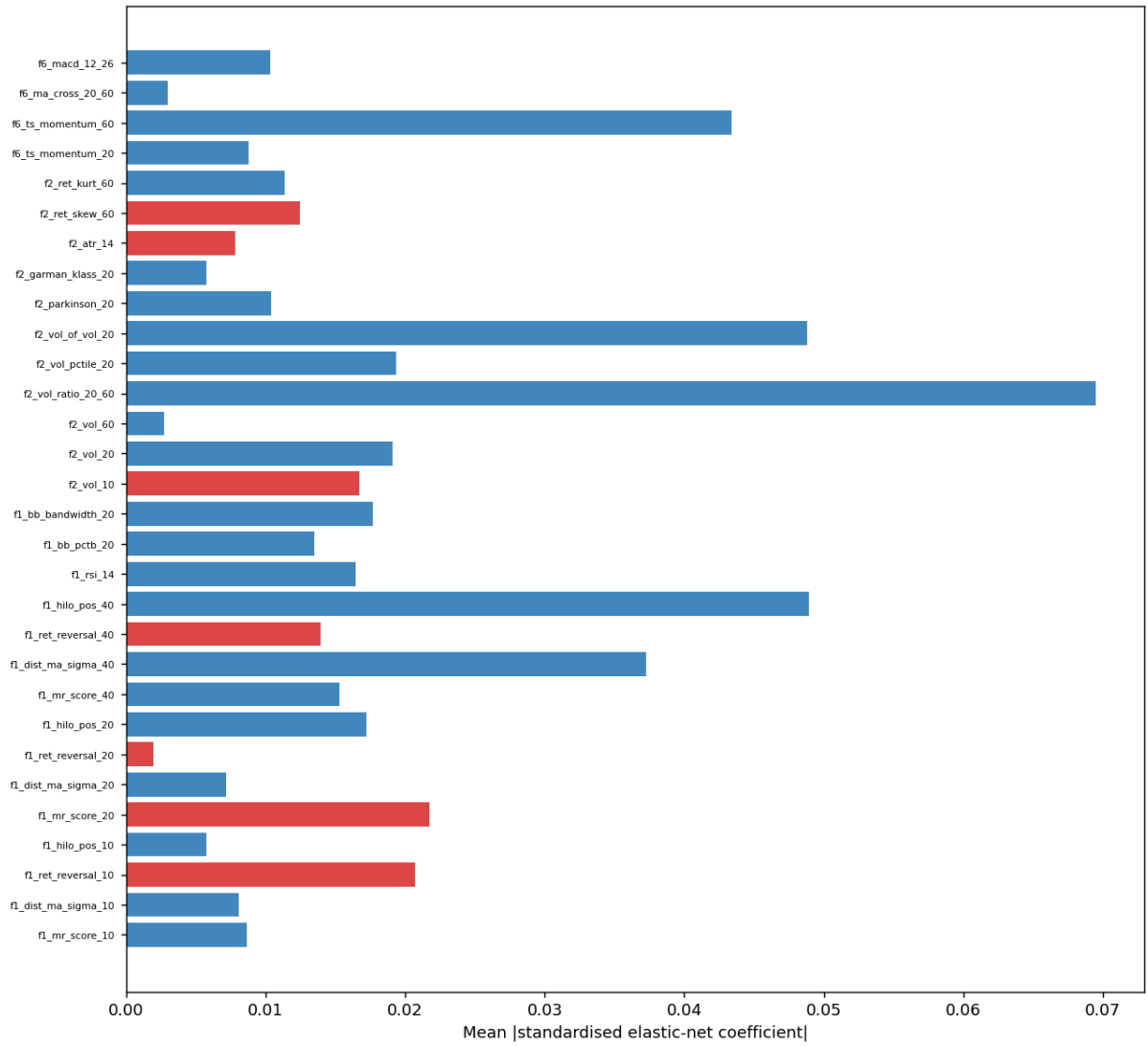




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
F5_signal	0.0292	0.0461	False
C6_f11_lowfreq_macro	0.0152	0.0230	False
C1_f1	0.0111	0.0239	False
C3_f2	0.0104	0.0212	False
C4_f7	0.0095	0.0126	False
C2_f6	0.0081	0.0227	False
C7_f1	0.0040	0.0077	False
C12_f11	0.0034	0.0234	False

rb1s — Global per-feature coefficient (top 30)
red=positive, blue=negative

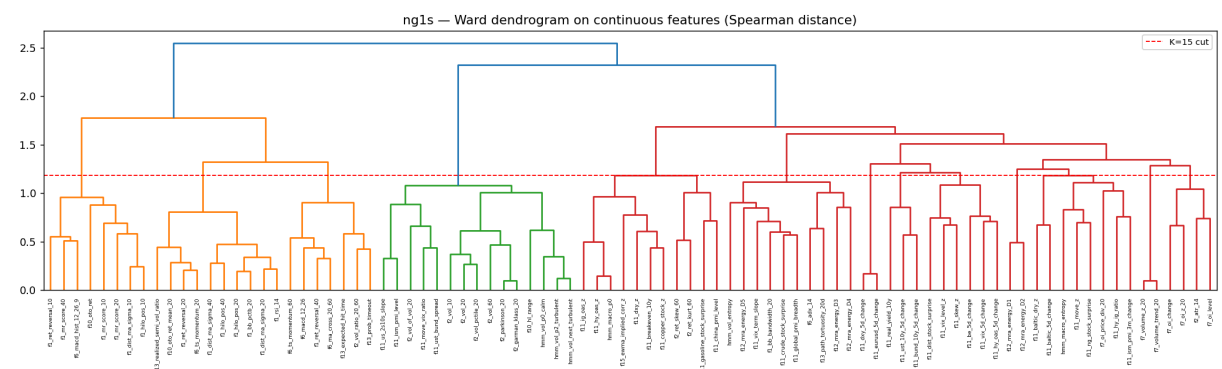


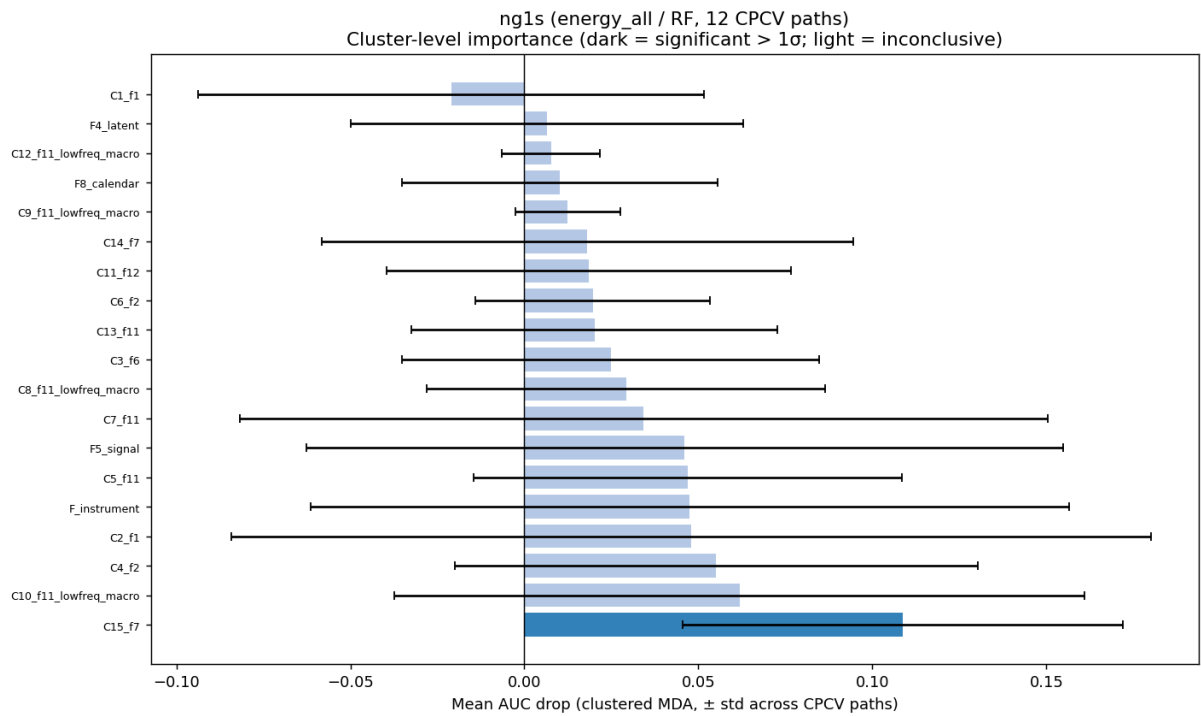
Top logistic coefficients

feature	coef_abs	coef_signed
f5_signal	0.2589	0.2589
f11_hy_oas_5d_change	0.1497	0.1497
f12_mra_energy_D2	0.0846	0.0846
f11_dist_stock_surprise	0.0838	0.0838
f11_gasoline_stock_surprise	0.0735	0.0735
hmm_vol_p0_calm	0.0720	-0.0720
f11_crude_stock_surprise	0.0707	0.0707
f2_vol_ratio_20_60	0.0695	-0.0695
f7_oi_level	0.0589	-0.0589
f11_dxy_z	0.0563	0.0563
f1_hilo_pos_40	0.0489	-0.0489
f2_vol_of_vol_20	0.0488	-0.0488
f7_oi_price_div_20	0.0474	-0.0474
f5_participation_20	0.0435	0.0435
f6_ts_momentum_60	0.0434	-0.0434

Henry Hub natural gas (NG) — ng1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci
ng1s	energy	energy_all	tree	rf	0.6576	0.1389	0.5187

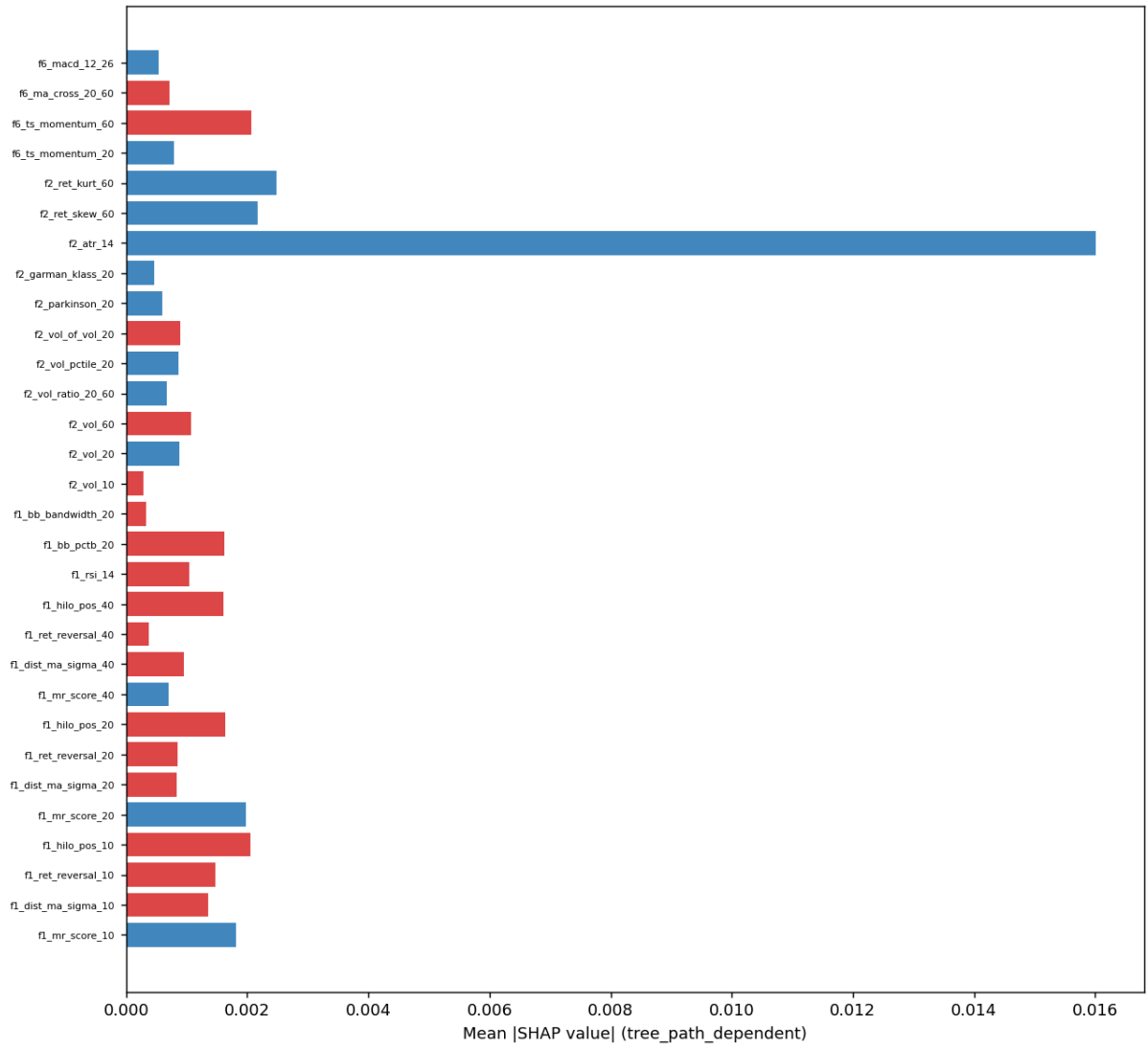




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C15_f7	0.1088	0.0633	True
C10_f11_lowfreq_macro	0.0618	0.0992	False
C4_f2	0.0551	0.0751	False
C2_f1	0.0480	0.1320	False
F_instrument	0.0476	0.1089	False
C5_f11	0.0468	0.0615	False
F5_signal	0.0461	0.1086	False
C7_f11	0.0343	0.1162	False

ng1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative



Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f7_oi_change	0.0191	0.0135	0.0680
f7_oi_level	0.0162	0.0162	0.0887
f2_atr_14	0.0160	-0.0160	0.0657
inst_rb1s	0.0152	0.0152	0.0860
f5_signal	0.0134	-0.0134	0.0305
hmm_vol_next_turbulent	0.0098	0.0098	0.0615
f5_participation_20	0.0091	0.0091	0.0362
hmm_vol_p2_turbulent	0.0061	0.0061	0.0332
f5_long_bias_20	0.0054	-0.0049	0.0225
hmm_vol_p0_calm	0.0045	0.0045	0.0381
f11_vix_term_slope	0.0042	0.0013	0.0186
f2_ret_kurt_60	0.0025	-0.0004	0.0092
hmm_macro_entropy	0.0023	0.0010	0.0111
f2_ret_skew_60	0.0022	-0.0019	0.0071
f15_ewma_implied_corr_z	0.0022	0.0010	0.0143

7. Strategy — pooled portfolio and per-class attribution

There is a single backtest, not one per class. The live strategy is one volatility-targeted, equally-weighted book over all eleven contracts: an EWMA(60) volatility estimate scales each position to a 10% annualised target, leverage is capped, returns lag the weights by a day, and Grinold-Kahn costs (a 2bp half-spread plus 10bp of turnover) are charged. The conventions are recorded in `results/strategy_eval/eval_summary.md`. Method A follows every signal; Method B trades only when the calibrated metamodel probability exceeds one half.

The headline table and cumulative-return figures below are the **pooled** result. The per-class table that follows is an **attribution** — this class's slice of the shared book — and not a standalone per-class Sharpe; with one pooled portfolio the instruments cannot be isolated into independent backtests.

```
In [8]: se = RESULTS / "strategy_eval"
table(load(se / "eval_summary.csv"), caption="Pooled portfolio – 00S
headline (eval_summary.csv)")
show(se / "cumulative_returns.png", width=860)
show(se / "per_asset_cumulative_returns.png", width=860)
```

```

show(se / "per_instrument_contribution.png", width=860)

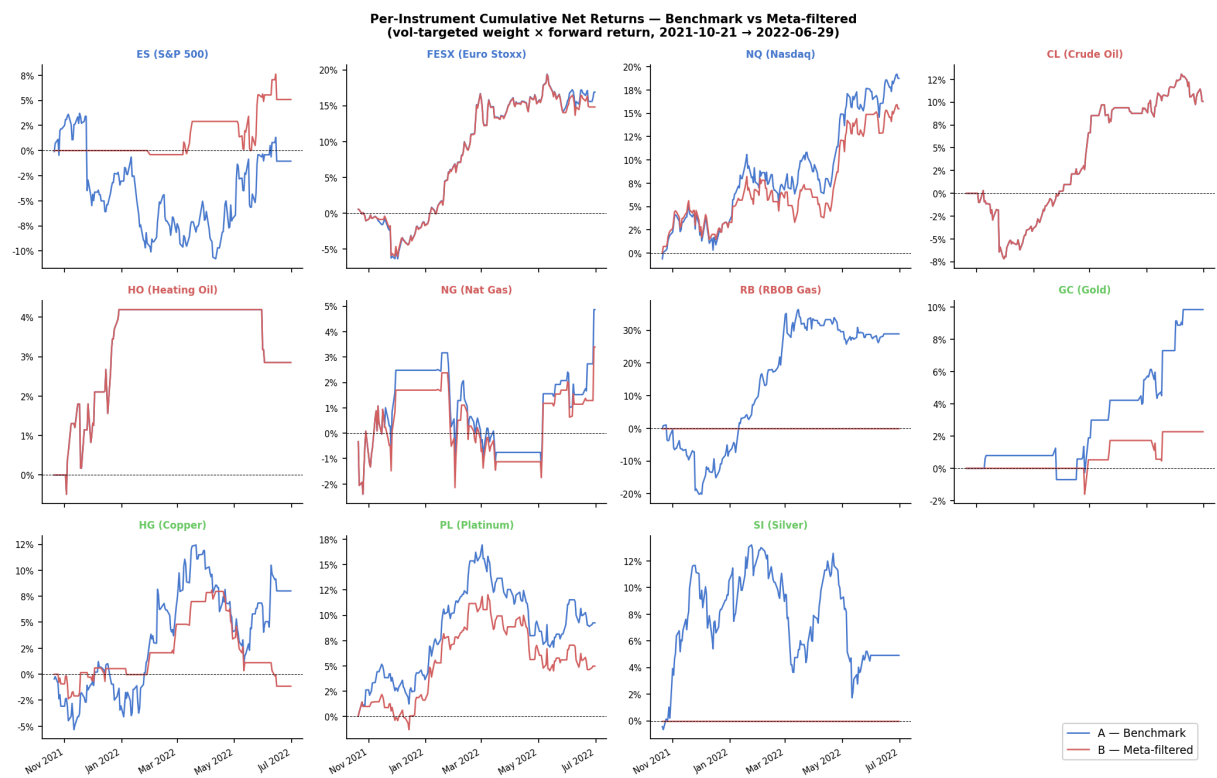
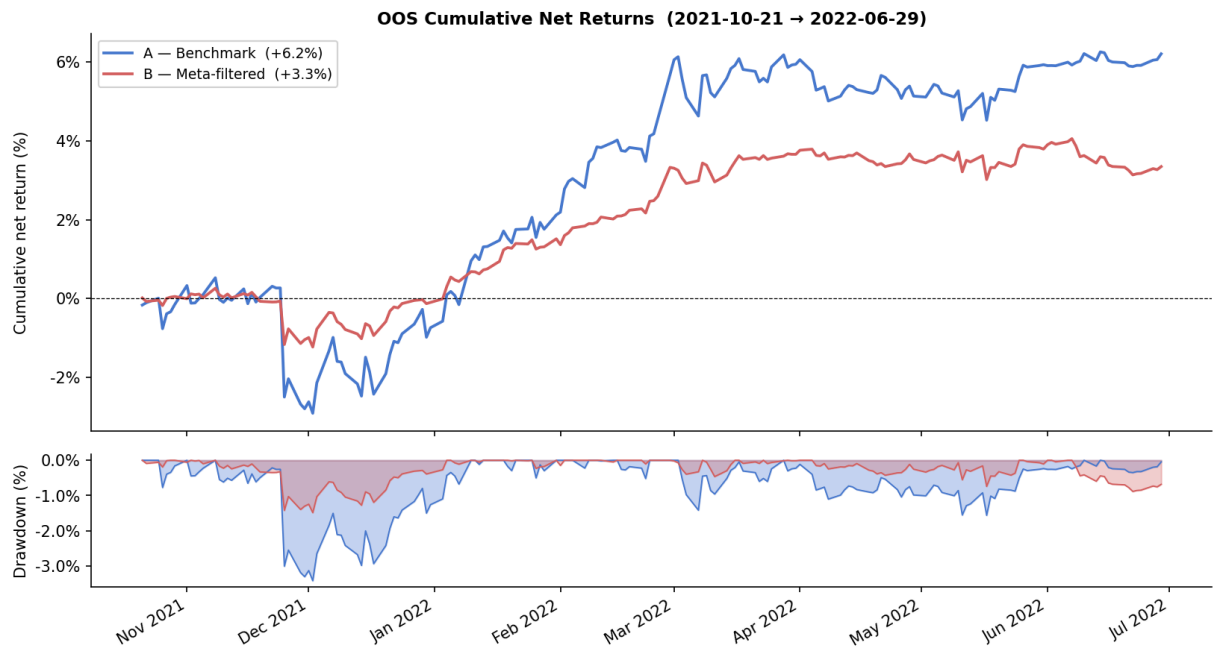
# --- Per-class attribution within the pooled 1/K book (with numeric
guards) ---
preds = load(OUT / "metamodel_predictions.csv")
assert not preds.empty, "metamodel_predictions.csv missing"
assert preds["calibrated_proba"].between(0, 1).all(), "calibrated_proba
outside [0, 1]"
assert set(INSTS) <= set(preds["instrument"].unique()), "class instruments
absent from predictions"

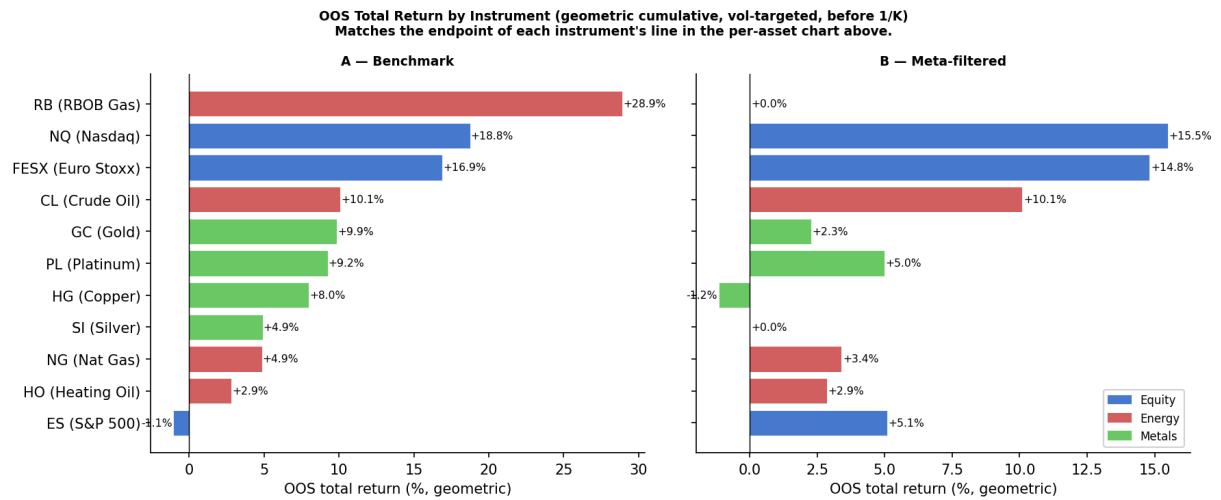
cls = preds[preds["instrument"].isin(INSTS)].copy()
cls["taken"] = cls["calibrated_proba"] > 0.5
attr = (cls.groupby("instrument")
        .apply(lambda g: pd.Series({
            "n_events": len(g),
            "events_taken": int(g["taken"].sum()),
            "mean_calib_proba": g["calibrated_proba"].mean(),
            "label1_share": g["bin"].mean(),
            "hit_rate_taken": g.loc[g["taken"], "bin"].mean() if
g["taken"].any() else float("nan"),
        }), include_groups=False)
        .reindex(INSTS).reset_index())
assert (attr["events_taken"] <= attr["n_events"]).all(), "events_taken
exceeds n_events"
table(attr, caption=f"{CLASS.title()} – attribution within the pooled book
(metamodel_predictions.csv)")

```

Pooled portfolio — OOS headline (eval_summary.csv)

	Metric	A Benchmark	B all_or_nothing	B SOPS
	Ann return	+8.62%	+4.65%	+4.65%
	Ann vol	6.10%	2.88%	2.88%
	Sharpe	1.412	1.614	1.614
	t-stat	1.19	1.36	1.36
	SR 95% CI low	-1.559	-1.286	-1.286
	SR 95% CI high	3.750	3.821	3.821
	PSR(SR*=0)	0.861	0.899	0.899
	Max DD	-3.42%	-1.49%	-1.49%
	Total cost (bps)	416.1	188.4	188.4
	Annual turnover	48.28	21.86	21.86
	Breakeven ½-spread	17.8 bps	21.3 bps	21.3 bps
	Events taken	100%	54%	54%





Energy — attribution within the pooled book (metamodel_predictions.csv)

instrument	n_events	events_taken	mean_calib_proba	label1_share	hit_rate_taken
cl1s	127.0000	127.0000	0.5869	0.6142	0.6142
ho1s	27.0000	27.0000	0.7159	0.6667	0.6667
rb1s	159.0000	0.0000	0.2329	0.3333	nan
ng1s	84.0000	78.0000	0.5406	0.5357	0.5128

8. Discussion

Energy is where the metamodel pays for itself. The complex contributes the bulk of the pooled book's out-of-sample return, with crude and the refined products doing the heavy lifting and natural gas contributing little once the calibrated filter has spoken. The cost of that return is modelling complexity: pooling, a special case for crude and a minimal-variant guard for gasoline. The pay-off justifies the bookkeeping, but the thin-instrument caveats should travel with the result.

References

- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ, Wiley. [Triple-barrier labelling ch. 3; average uniqueness ch. 4; purged and combinatorial cross-validation ch. 7.]
- Project methodology notes (this repository): `reports/harry/00-context.md`, `01-signal-direction.md`, `02-labels.md`, `03-features.md`, `03-6-macro-features.md`, and `reports/missing-data-report.md`.
- Strategy conventions: `results/strategy_eval/eval_summary.md` (StrategyWeights lecture, slides 38–43).
- Course materials: *Systematic Trading Strategies with Machine Learning* (T3.03), lectures L1–L8.